

//Entrada

```
< if proximo == -1  
    proximo = i;  
    else  
        cola.push(i); >
```

```
< await proximo == i >
```

//Imprimir

```
Imprimir()  
< if cola.empty():  
    proximo = -1;  
    else
```

//salida

```
    proximo = cola.pop(); >
```

Práctico 2: semáforos

Sintaxis de semáforos

La declaración siempre tendrá una inicialización obligatoria

sem mutex = 1 // semáforo binario p/ V mutex

sem espera[5] = ([5] 1); // array de 5 semáforos, todos inicializados en 1

Operaciones (únicas 2 y atómicas)

P(s) → < await (s > 0) s = s - 1; > // se demanda s, s > 0; s, no resta 1

V(s) → < s = s + 1; > // nunca se demanda, solo suma 1

Hay 2 usos de un semáforo, q' conviene distinguir siempre

1) Semáforo de exclusión mutua (mutex): inicializado en 1. Protege una SC p/ P(mutex)

V(mutex). Garantiza q' un único proceso esté adentro a la vez.

2) Semáforo de sincronización/contador de recursos: inicializado en 0 (o en la

cant. de recursos disponibles). No protege una SC crítica: un proceso hace V

p/ "avisar" o "liberar un recurso", otro proceso hace P p/ "esperar el recurso"

o "tomar el recurso" ~~proceso impide otros~~. El P y el V cas. siempre están en procesos <>.

< > y < await > vs semáforos

< S > y < await B; S > son una forma de especificar sincronización: dicen "quiero q' esto se comporte como atómico" o "quiero esperar hasta q' se cumple B y ahí esperar S sin q' nada interfiera". Admiten cualquier condx B y cualquier secuencia S, pero x eso mismo no se puede esperar tal cual en una máquina real - no hay forma eficiente de esperar una condx sin quedar consumiendo CPU en un loop.

El semáforo es la herramienta real, la q' s. se puede implementar: es un contador entero (número negativo) + una cola de procesos bloqueados p/ solo dos operaciones posibles. V < definir P(s) es un caso particular de < await B; S >, donde B es siempre "s > 0" y S es siempre "s = s - 1" / V(s) es un caso particular de < s > donde S es siempre "s = s + 1". O sea, el semáforo no es otra cosa <>, es la misma idea pero restringida a una sola forma de condx (con contador mayor q' 0), q' es justo lo q' hace implementable de verdad.

1) a) único proceso = Personero.
 recurso = detector de metales.
 semáforo/sincronización = único semáforo p/ acceder al detector.

b) sem mutex = 1;

1 detector

```
process Personero [i = 0..N] {
  P(mutex);
  pasarPorDetector();
  V(mutex);
}
```

c) sem mutex = 3

3 detectores

```
process Personero [i = 0..N] {
  P(mutex);
  pasarPorDetector();
  V(mutex);
}
```

d) sem detectoresDisponibles = 3;

```
process Personero [i = 0..N] {
  int veces;
  veces = numeroAlleenbro();
  for (j = 1..veces)
    P(detectoresDisponibles);
    pasarPorDetector();
    V(detectoresDisponibles);
}
```

2) a) historial[N]; sem mutex = 1;

```
process ProcesoControl [i = 0..3] {
  tam = N/4;
  inicio = i * tam;
  fin = inicio + tam - 1;
  for (j = inicio..fin) {
    if (historial[j].gravedad == 3) {
      P(mutex);
      imprimirEnPantalla(historial[j].id);
      V(mutex);
    }
  }
}
```

mutex = único semáforo

① si uso semáforo el/los recursos de p/ ser mutex.

Usar otro nombre p/ mayor legibilidad
 ej. detectoresDisponibles

b) $\text{sem mutex}[4] = (1, 1), \quad \text{fallos}[4] = (1, 0);$
 $\text{historial}[N];$
 $\text{process ProcesoControl } [i: 0..3] \{$
 $\text{tem} = N/4;$
 $\text{inicio} = K \cdot \text{tem};$
 $\text{fin} = \text{inicio} + \text{tem} - 1;$

$\text{for } (j = \text{inicio} \dots \text{fin}) \{$
 $g = \text{historial}[j]. \text{ gravedad};$
 $P(\text{mutex}[g]);$
 $\text{fallos}[g]++;$
 $V(\text{mutex}[g]);$
 $\}$

c) $\text{historial}[N]; \text{fallos}[4] = (1, 0);$
 $\text{process ProcesoControl } [i: 0..3] \{$
 $\text{for } (j = 0 \dots N) \{$
 $\text{if } (\text{historial}[j]. \text{ gravedad} == i) \{$
 $\text{fallos}[i]++;$
 $\}$
 $\}$
 $\}$

(1) revisar, de sin sonápros?
 no se pisa "g" / u
 usó $\leftrightarrow$ indirec.

está OK! no era necesario

3) $\text{queue cola} = \text{new } \text{Cola}(5);$
 $\text{sem mc} = 1, \quad \text{sem recursosDisponibles} = 5;$
 $\text{process } \text{Proceso } [id: 0..P-1] \{$
 $P(\text{recursosDisponibles});$
 $P(\text{mc});$
 $\text{Recurso } r = \text{cola.pop } (); \quad \text{cola.push}(r);$
 $V(\text{mc}); \text{ usar}(r); P(\text{mc}); \quad V(\text{mc}); V(\text{recursosDisponibles});$

4) Es correcta pero no del todo eficiente xq. primero evalúa el recurso y luego prioridades, debería de ser al revés así en caso de q. el proceso quede bloqueado sea xq. no hay rpo.

5) a) $\text{sem libres} = N$ // contenedores vacíos disponibles, ahora lleno de espacio
 $\text{sem ocupado} = 0;$ // paquetes ya depositados. lista p/ entrega
 $\text{int pri-voz} = 0;$ // proximo contenedor dentro el Repartidor deja el paquete
 $\text{int pri-ocupado} = 0;$ // " Entregador va a sacar un paquete
 $\text{contenedor buffer}[N];$

$\text{Process Repartidor } \{$
 $\text{while } (true) \{$
 $\text{preparar Paquete}();$
 $P(\text{libres});$
 $\text{buffer}[pri-voz] = \text{paquete};$
 $\text{pri-voz} = (\text{pri-voz} + 1) \bmod N;$
 $V(\text{ocupado});$
 $\}$
 $\}$

$\text{Process Entregador } \{$
 $\text{while } (true) \{$
 $P(\text{ocupado});$
 $\text{paquete} = \text{buffer}[pri-ocupado];$
 $\text{pri-ocupado} = (\text{pri-ocupado} + 1) \bmod N;$
 $V(\text{libres});$
 $\text{realiza Entrega};$
 $\}$
 $\}$

b) $\text{sem libre} = N$ $\text{sem ocupado} = 0$;
 ~~$\text{sem ocupado} = 0$~~ ; $\text{sem mutex} = 1$; //protege pri-var
 $\text{int pri-var} = 0$;
 $\text{int pri-ocupado} = 0$;
 $\text{contenedor buffer [N]}$;

Process Preparador $[i = 1..N]$ {
 while (true) {
 preparar Paquete ();
 P(libre); P(mutex);
 $\text{buffer[pri-var]} = \text{paquete}$
 $\text{pri-var} = (\text{pri-var} + 1) \text{ MOD } N$;
 V(mutex);
 V(ocupado);
 }

Process Entregador {
 while (true) {
 P(ocupado);
 $\text{paquete} = \text{buffer[pri-ocupado]}$;
 $\text{pri-ocupado} = (\text{pri-ocupado} + 1) \text{ mod } N$;
 V(libre);
 }

c) $\text{sem libre} = N$; $\text{sem ocupado} = 0$;
 $\text{sem mutex} = 1$;
 $\text{int pri-var} = 0$;
 $\text{int pri-ocupado} = 0$;
 $\text{contenedor buffer [N]}$;

process Preparador {
 while (true) {
 preparar Paquete ();
 P(libre);
 $\text{buffer[pri-var]} = \text{paquete}$;
 $\text{pri-var} = (\text{pri-var} + 1) \text{ MOD } N$;
 V(ocupado);
 }

process Entregador $[i = 1..N]$ {
 while (true) {
 P(ocupado);
 P(mutex);
 $\text{paquete} = \text{buffer[pri-ocupado]}$;
 $\text{pri-ocupado} = (\text{pri-ocupado} + 1) \text{ mod } N$;
 V(mutex);
 V(libre);
 }

d) $\text{sem libre} = N$; $\text{sem ocupado} = 0$; $\text{sem mutex P} = 0$; $\text{mutex E} = 1$; $\text{int pri-var} = 0$;
 $\text{int pri-ocupado} = 0$; $\text{contenedor buffer [N]}$;

process Preparador $[i = 1..F]$ {
 while (true) {
 preparar Paquete ();
 P(libre);
 P(mutex P);
 $\text{buffer[pri-var]} = \text{paquete}$;
 $\text{pri-var} = (\text{pri-var} + 1) \text{ MOD } N$;
 V(mutex P);
 V(ocupado);
 }

process Entregador $[j = 1..S]$ {
 while (true) {
 P(ocupado);
 P(mutex E);
 $\text{paquete} = \text{buffer[pri-ocupado]}$;
 $\text{pri-ocupado} = (\text{pri-ocupado} + 1) \text{ mod } N$;
 V(mutex E);
 V(libre);
 }

NOTA

```

c) a) sem opaco = 1;
process Person [id: 1..N] {
    R(opaco);
    Imprimir Documento ();
    V(opaco);
}

```

```

b) cola C; sem mutex = 1; espera[N] = (C[N] 0);
boolean libe = true;

```

```

process Person [id: 1..N] {
    int aux;
    P(mutex);
    if (libe) {
        libe = false;
        V(mutex);
    } else {
        push(C, id);
        V(mutex);
        P(espera[id]);
        Imprimir Documento ();
        P(mutex);
        if (empty(C)) libe = true;
        else {
            pop(C, aux); V(espera[aux]);
        }
        V(mutex);
    }
}

```

```

c) sem turno[N+1] = (N+1 0);
turno[1] = 1;
process Person [id: 1..N] {
    P(turno[id]);
    Imprimir Documento ();
    if (id < N) V(turno[id+1]);
}

```

```

d) sem turno[N+1] = (N+1 0); sem pedidos = 0; sem terminado = 0;
cola C; sem mutex = 1;

```

```

process Person [id: 1..N] {
    P(mutex); push(C, id);
    V(mutex);
    V(pedidos);
    P(turno[id]);
    Imprimir Documento ();
    V(terminado);
}

```

```

process Coordenador {
    int aux;
    while (true);
    P(pedidos);
    P(mutex);
    pop(C, aux);
    V(mutex);
    V(turno[aux]);
    P(terminado);
}

```

e) colz C; colz colImpresoras; sem mutex P = 1, pedidos = 0;
 sem mutex I = 1, implibres = 5;
 sem turno [N] = (N) 0;
 int impresoraAsignada[N];

```

process Persona[id: 1..N]
P(mutex P); push(C[id], V(mutex P);
V(pedidos);
P(turno[id]);
Impresa(ImpresoraAsignada[id], documento);
P(mutex I); push(ColaImpresoras, impresoraAsignada[id]);
V(mutex I);
V(implibres);
}

```

```

process Coordinador
int persona, imp;
while (true)
{
  P(pedidos);
  P(implibres);
  P(mutex P); pop(C[persona]); V(mutex P);
  P(mutex I);
  pop(ColaImpresoras, imp); V(mutex I);
  impresoraAsignada[persona] = imp;
  V(turno[persona]);
}
}

```

f) int contadorGrupo[10] = ([10] 0); sem mutexGrupo[10] = ([10] 1);
 int ordenGlobal = 0; sem mutexOrden = 1; int puntero[10]; sem esperaGrupo[10] = ([10] 0);

```

process Alumno[i: 1..50]
  int aux = elegir();
  realizarTarea(aux);
  P(mutexGrupo[i]);
  contadorGrupo[aux] = contadorGrupo[i] + 1;
  if (contadorGrupo[aux] == 5)
  {
    P(mutexOrden);
    ordenGlobal = ordenGlobal + 1;
    puntero[aux] = ordenGlobal;
    V(mutexOrden);
    for (j = 1..5) V(esperaGrupo(aux));
  }
  V(mutexGrupo[i]);
  P(esperaGrupo[i]);
}

```

process Profesor { N/A

8) a) $\text{int } \text{maximoProducto} = 1; \text{ int } \text{cantMaximo} = 0; \text{ int } \text{piezasRestantes} = T;$
 $\text{sem } (\text{Llegaron}) = 0; \text{ sem } \text{Lleg} = \text{int } \text{barra} = \text{sem } \text{mutexPiezas} = 1;$
 $\text{sem } \text{barra} = 0; \text{ int } \text{Llegados} = 0; \text{ sem } \text{mutexLlegados} = 1; \text{ int } \text{piezasHechas}[E] = (E=0);$

```

process Empleado [i = 1..E] {
    int P(mutexLlegados);
    Llegados++;
    if (Llegados == E) {
        for (j = 1..E) V(barra);
    }
    V(mutexLlegados);
    P(barra);
    P(mutexPiezas);
    while (piezasRestantes > 0) {
        P(mutexPiezas);
        piezasRestantes = piezasRestantes - 1;
        fabricaPiezas;
        piezasHechas[id]++;
        P(mutexPiezas);
    }
    V(mutexPiezas);
    P(mutexMax);
    if (piezasHechas[id] == MaxPiezas) {
        maxPiezas = piezasHechas[id];
        emplazamiento = id;
    }
    V(mutexMax);
}

```

Espero llegada de todos.

b) idem a) contempla cuolg. valr c/ "int piezasRestantes = T".

9) $\text{sem } \text{marcosLibres} = 30; \text{ sem } \text{marcosOcupados} = 0;$
 $\text{sem } \text{vidriosLibres} = 50; \text{ sem } \text{vidriosOcupados} = 0;$
 $\text{sem } \text{mutexV} = 1; \text{ sem } \text{mutexM} = 1;$

```

process Carpintero [i = 1..4] {
    while (true) {
        generaMarco();
        P(mutexLibre);
        P(mutexM);
        depositoMarco[pi_vacio_marco] = marco;
        pi_vacio_marco = (pi_vacio_marco + 1) mod 30;
        V(mutexM);
        V(marcosOcupados);
    }
}

```

```

process Vidriero {
    while (true) {
        generaVidrio();
        P(mutexLibre);
        P(mutexV);
        depositoVidrio[pi_vacio_marco] = vidrio;
        pi_vacio_marco = (pi_vacio_vidrio + 1) mod 50;
        V(mutexV);
        V(marcosOcupados);
    }
}

```

```

int pi_vacio_marco = 0;
int pi_ocupado_marco;
marco deposito_marco[30];
int pi_vacio_vidrio = 0;
int pi_ocupado_vidrio = 0;
vidrio deposito_vidrio[50];

```

```

process Almacenes [j = 1..2] {
    while (true) {
        P(marcosOcupados);
        P(vidriosOcupados);
        P(mutexM);
        marco = marco_deposito[pi_ocupado_marco];
        pi_ocupado_marco = (pi_ocupado_marco + 1) mod 30;
        V(mutexM);
        P(mutexV);
        vidrio = vidrio_deposito[pi_ocupado_vidrio];
        pi_ocupado_vidrio = (pi_ocupado_vidrio + 1) mod 50;
        V(mutexV);
    }
}

```

NOTA

sem mutex = 1; cola turnos; sem pedidos = 0;

10) a) sem camionesTrigo [1..T];
sem camionesMaiz [1..M];

sem total = 1;
sem trigo = 5;
sem maiz = 5;

```
process CamionTrigo [i: 1..T] {
  P(mutex);
  turnos.push(i, tipo);
  V(mutex);
  V(pedidos);
  P(camionesTrigo[i]);
  descargaCamion();
  V(total);
  V(trigo);
}
```

```
process CamionMaiz [j: 1..M] {
  P(mutex);
  turnos.push(j, tipo);
  V(mutex);
  V(pedidos);
  P(camionesMaiz[j]);
  descargaCamion();
  V(total);
  V(maiz);
}
```

```
process Coordinador {
  tipo, tipo; ind id;
  for (K=1; K ≤ T+M; K++) {
    P(pedidos);
    P(mutex);
    (tipo, id) = turnos.pop();
    V(mutex);
    if (tipo == TRIGO) P(trigo) else P(maiz);
    P(total);
    if (tipo == TRIGO) V(camionesTrigo[id]); else
      V(camionesMaiz[id]);
  }
}
```

b) sem trigo = 5; sem maiz = 5; sem total = 1;

```
process trigo [i: 1..T] {
  P(trigo);
  P(total);
  descarga();
  V(total);
  V(trigo);
}
```

```
process maiz [j: 1..M] {
  P(maiz);
  P(total);
  descarga();
  V(total);
  V(maiz);
}
```

11) ind cant = 0; sem mutex = 1; cola;

```
process Enfermero {
  for (K=1; K ≤ 5; K++) {
    P(llegados);
    for (i=1; i ≤ 5; i++) {
      P(mutex);
      i = cola.pop();
      V(mutex);
      V(turno[i]);
      llamarPersona(i);
    }
  }
}
```

sem llegados = 0;
sem turno[50] = (30(0));

```
process Persona [i: 1..50] {
  P(mutex);
  cant++;
  cola.push(i);
  if (cant % 5 == 0) {
    V(llegados);
  }
  V(mutex);
  P(turno[i]);
}
```

```

12) sem mutex Recep = 1, hay Pedido = 0;
cola pedidos;
int puesto Asignado [150];
sem aviso recepcion [150] ([150] 0);
sem mutex Esp = 1;
int operando [3] = {0, 0, 0};
sem mutex cola [3] = ([3] 1);
cola colaPrestos [3];
sem pacientes [3] = ([3] 0);
sem aviso Enfermera [150] = ([150] 0);

```

```

proceso Pajero [i = 1..150] {

```

```

    int p;
    P(mutex Recep);
    pedidos.push(i);
    V(mutex Recep);
    V(hay Pedido);
    P(aviso recepcion [i]);
    p = puesto asignado [i];
    P(mutex cola [p]);
    colaPrestos[p].push(i);
    V(mutex cola [p]);
    V(pacientes [p]);
    P(aviso Enfermera [i]);
}

```

```

proceso Recepcionista

```

```

    while (true) {
        P(hay Pedido);
        P(mutex Recep);
        id = pedidos.pop();
        V(mutex Recep);
        P(mutex Esp);
        p = 0;
        for (k = 1; k <= 2; k++) {
            if (operando[k] < operando[p]) p = k;
            operando[p]++;
            V(mutex Esp);
            puesto Asignado [id] = p;
            V(aviso recepcion [id]);
        }
    }
}

```

```

proceso Enfermera [p = 1..2] {

```

```

    int id;
    while (true) {
        P(pacientes [p]);
        P(mutex cola [p]);
        id = colaPrestos[p].pop();
        V(mutex cola [p]);
    }
}

```

NOTA